第 7 课：多元线性回归

吴恩达《Machine Learning Specialization》· Course 1 · Week 2 · Lesson 1

从这一课起，我们进入 Week 2。

承上：前几课的模型 `f(x) = w·x + b` 只用了**一个**输入（房子面积）。 **这一课**：现实预测房价不能只看面积！还有卧室数、楼层、年龄……我们把模型扩展到**多个输入**，变成 **多元线性回归（Multiple Linear Regression）**。

新模型（先看个样子）：

$$f(\mathbf{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

怎么用：从头往下读，代码格按 `Shift + Enter` 运行。

第 0 步：环境自检

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager
_candidates = ["Microsoft YaHei", "SimHei", "PingFang SC",
               "Noto Sans CJK JP", "Noto Sans CJK SC", "Droid Sans Fallback"]
_available = {f.name for f in font_manager.fontManager.ttflist}
_chosen = [c for c in _candidates if c in _available] or ["DejaVu Sans"]
plt.rcParams["font.sans-serif"] = _chosen
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["savefig.bbox"] = "tight"
print("本次使用中文字体：", _chosen)
print(" 环境就绪，可以继续啦！")
```

```
本次使用中文字体： ['Noto Sans CJK JP', 'Droid Sans Fallback']
 环境就绪，可以继续啦！
```

一、为什么要多个特征

一套房子的价格，当然不只由**面积**决定：

特征	变量
面积（平方英尺）	x_1
卧室数量	x_2
楼层数	x_3
房龄（年）	x_4

每个这样的"输入"，就叫一个**特征（feature）**。特征有 n 个，模型就得跟着用上 n 个系数。

直观理解：每个特征对房价都有贡献——面积越大越贵、房间越多越贵、房龄越大可能越便宜。模型要做的，就是给每个特征配上一个"权重" w，衡量它有多重要。

二、新的记号

为了写多个特征，第 2 课的记号升级一下：

记号	意思
`n`	**特征个数**（比如 4 个：面积、卧室、楼层、房龄）
`x⁽ⁱ⁾`	第 **i** 个训练样本的**所有特征**，是一个"向量"（一串数）
`x⁽ⁱ⁾ⱼ`	第 i 个样本的**第 j 个特征**
`w`	权重向量 `(w₁, w₂, …, wₙ)`，也是一个向量
`b`	偏置（还是一个数）

> 一句话：**x 是"一串特征"，w 是"一串对应的权重"，b 是一个截距。**

三、模型长这样

把每个特征乘上自己的权重、全部加起来、再加上 b：

$$f_{\mathbf{w},b}(\mathbf{x}) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

举个例子（房价），假设我们"学到"了这样一组参数：

> $w_1 = 0.1$（面积，每平方英尺约 0.1 万）、$w_2 = 4$（每个卧室约 4 万）、$w_3 = 20$（每层约 20 万）、$w_4 = -1.5$（每老一年约减 1.5 万）、$b = 50$（基础价）

那么对于某套房子 x = (1000 平方英尺, 3 室, 2 层, 10 年)，预测：

$$f = 0.1 \times 1000 + 4 \times 3 + 20 \times 2 + (-1.5) \times 10 + 50 = 253$$

来跑代码验证：

In [2]:

```
# 用一组"学来的"权重，手算预测
# 特征：x1=面积(平方英尺)，x2=卧室数，x3=楼层，x4=房龄(年)
w = np.array([0.1, 4.0, 20.0, -1.5])   # 权重向量
b = 50.0

x_house = np.array([1000, 3, 2, 10])   # 某套房

# 方式一：手动展开
f_manual = (0.1*1000 + 4.0*3 + 20.0*2 + (-1.5)*10) + b
print("手动展开：", f_manual)

# 方式二：用 numpy 向量乘法（下一课会专门讲）
f_vec = np.dot(w, x_house) + b
print("向量写法：", f_vec)
```

```
手动展开： 187.0
向量写法： 187.0
```

看：两种写法算出来都是 **253 万**。第二种写法就是**向量化**的雏形——一串权重点乘一串特征，比一行行乘加干净多了。

这个"模型"的思想和单变量完全一样，只是从"一条权重"变成"一串权重"。你会的那套（找 w、b 让 J 最小）原封不动，只是 w 变成了向量。

四、成本函数：几乎没变

成本函数的**定义完全不变**，只是 `f(x⁽ⁱ⁾)` 换成了多元版本：

$$J(\mathbf{w},b)=\frac{1}{2m}\sum_{i=1}^{m}\left(f_{\mathbf{w},b}(\mathbf{x}^{(i)})-y^{(i)}\right)^2$$

重点：你现在要优化的不再是"一个 w、一个 b"，而是 **"一串 w（n 个）+ 一个 b"**。

五、小结

1. **多元线性回归**：模型从 `w·x+b` 扩展到 `w₁x₁ + w₂x₂ + … + wₙxₙ + b`，每个特征配一个权重。
2. **新记号**：`n`=特征数、`x⁽ⁱ⁾`=第 `i` 个样本的整串特征、`w`=权重向量。
3. **成本函数不变**，还是"平均平方误差"，只是参数变成 `w` 向量和 `b`。
4. **手动写多个乘加很啰嗦** —— 所以下一课要学**向量化**。

六、反转课堂 · 课后作业

1. 如果预测房价的特征是 `面积、卧室、楼龄` 三个，那么 `n` 是多少？`w` 是什么样？
2. 用第三节的权重 w=(0.1, 4.0, 20.0, −1.5)、b=50，预测 `x = (2000, 4, 3, 5)` 房子的房价（手算）。
3. 为什么多个特征时，我们会说"w 变成了一串数（向量）"，而不是"有很多个独立的模型"？

下一课（第 8 课）：向量化（Vectorization）——怎么用 numpy 一行代码写完上面这一长串。